

MÓDULO 3: LIMPIEZA Y TRANSFORMACIÓN DE DATOS (ETL)

Proyecto: Análisis ETL - Entidad Financiera

Prerequisito: MÓDULO 2 completado (datos cargados)

OBJETIVO

Implementar el proceso ETL completo:

- **Extract** (Extraer): Ya hecho en Módulo 2
- **Transform** (Transformar): Limpiar, corregir y enriquecer datos
- **Load** (Cargar): Guardar datos limpios en CSV

PASO 1: DETECTAR PROBLEMAS EN LOS DATOS

► 1.1 Verificar valores nulos

Nueva celda en el notebook:

```
# ANÁLISIS DE CALIDAD DE DATOS
print('=' * 80)
print('VERIFICACIÓN DE VALORES NULOS')
print('=' * 80)

print('\n📊 DATASET DE CLIENTES:')
print(df_customers.isnull().sum())

print('\n📊 DATASET DE VENTAS:')
print(df_sales.isnull().sum())
```

Salida:

```
=====
VERIFICACIÓN DE VALORES NULOS
=====

📊 DATASET DE CLIENTES:
customer_id      0
age             119 ⚠️
gender           0
location         0
membership_years 0
dtype: int64

📊 DATASET DE VENTAS:
transaction_id   0
customer_id      0
invoice_date     0
invoice_no       0
product_category 0
quantity         0
price            0
payment_method   0
shopping_mall    0
dtype: int64
```

Interpretación:

- ☒ Ventas: Sin valores nulos
- ⚠️ Clientes: 119 nulos en age (0.12% del total)

► 1.2 Verificar duplicados

Nueva celda:

```
# Verificar registros duplicados
print('\n🔍 VERIFICACIÓN DE DUPLICADOS:')
print(f'Clientes duplicados: {df_customers.duplicated().sum()}')
print(f'Ventas duplicadas: {df_sales.duplicated().sum()}')
```

Salida esperada:

```
🔍 VERIFICACIÓN DE DUPLICADOS:
Clientes duplicados: 0
Ventas duplicadas: 0
```



PASO 2: LIMPIEZA DE DATOS

► 2.1 Manejo de valores nulos en edad

Opciones para manejar nulos:

Opción	Ventaja	Desventaja
Eliminar filas	Simple, datos 100% completos	Pérdida de 119 transacciones
Imputar con media	Conserva todos los registros	Introduce sesgo
Imputar con mediana	Robusto a outliers	Menos preciso
Dejar nulos	Datos originales intactos	Puede causar errores

Decisión: Eliminar filas (solo 0.12% de pérdida).

Nueva celda:

```
# LIMPIEZA: Eliminar registros con edad nula
print('=' * 80)
print('LIMPIEZA DE DATOS')
print('=' * 80)

print(f'\n📊 Registros antes de limpieza: {len(df_customers):,}')

# Eliminar nulos
df_customers_clean = df_customers.dropna(subset=['age'])

print(f'📊 Registros después de limpieza: {len(df_customers_clean):,}')
print(f'✗ Registros eliminados: {len(df_customers) - len(df_customers_clean)}')
print(f'✅ Porcentaje conservado: {(len(df_customers_clean)/len(df_customers))*100:.2f}%')

# Verificar
print(f'\n✅ Nulos restantes en age: {df_customers_clean["age"].isnull().sum()}')
```

¿Qué hace `.dropna()` ?

- `subset=['age']` : Solo considera columna `age`
- Elimina filas donde `age` es `NaN`
- Retorna nuevo DataFrame sin modificar el original

Salida:

```
=====
LIMPIEZA DE DATOS
=====

📊 Registros antes de limpieza: 99,457
📊 Registros después de limpieza: 99,338
✗ Registros eliminados: 119
✅ Porcentaje conservado: 99.88%

✅ Nulos restantes en age: 0
```

► 2.2 Fusionar datasets

¿Por qué fusionar?

- Cada venta necesita información del cliente
- `customer_id` es la clave que conecta ambas tablas

Nueva celda:

```
# FUSIÓN DE DATASETS
print('\n' + '=' * 80)
print('FUSIONANDO DATASETS')
print('=' * 80)

# Merge usando customer_id como clave
df_merged = pd.merge(
    df_sales,          # Tabla izquierda (ventas)
    df_customers_clean, # Tabla derecha (clientes)
    on='customer_id',  # Columna común
    how='inner'         # Solo registros que coinciden
)

print(f'\n✅ Fusión completada')
print(f'📊 Registros en dataset fusionado: {len(df_merged):,}')
print(f'📋 Columnas totales: {df_merged.shape[1]}')
print(f'\n📋 Nuevas columnas agregadas desde clientes:')
print([col for col in df_merged.columns if col in df_customers_clean.columns and col != 'customer_id'])
```

Tipos de merge:

Tipo	Descripción	Resultado
<code>inner</code>	Solo coincidencias	Registros con customer_id en ambas tablas
<code>left</code>	Todos de izquierda	Todas las ventas, clientes pueden ser null
<code>right</code>	Todos de derecha	Todos los clientes, ventas pueden ser null
<code>outer</code>	Todos de ambas	Union completa

Salida:

```
=====
FUSIONANDO DATASETS
=====

[✓] Fusión completada
[📊] Registros en dataset fusionado: 99,338
[📋] Columnas totales: 12

[📋] Nuevas columnas agregadas desde clientes:
['age', 'gender', 'location', 'membership_years']
```

Columnas finales:

```
De ventas: transaction_id, customer_id, invoice_date, invoice_no,
           product_category, quantity, price, payment_method, shopping_mall
De clientes: age, gender, location, membership_years
```



PASO 3: TRANSFORMACIONES DE DATOS

► 3.1 Convertir fecha a formato datetime

Problema: `invoice_date` es texto (`object`), no fecha.

Nueva celda:

```
# TRANSFORMACIÓN: Convertir fechas
print('\n' + '=' * 80)
print('TRANSFORMACIÓN DE FECHAS')
print('=' * 80)

print(f'\nFormato original: {df_merged["invoice_date"].dtype}')
print(f'Ejemplo: {df_merged["invoice_date"].iloc[0]}')

# Convertir a datetime
df_merged['invoice_date'] = pd.to_datetime(
    df_merged['invoice_date'],
    format='%d-%m-%Y' # día-mes-año
)

print(f'\n✅ Formato transformado: {df_merged["invoice_date"].dtype}')
print(f'Ejemplo: {df_merged["invoice_date"].iloc[0]}')
print(f'\nRango de fechas: {df_merged["invoice_date"].min()} a {df_merged["invoice_date"].max()}'
```

¿Qué hace `pd.to_datetime()` ?

- Convierte texto a tipo `datetime64[ns]`
- `format='%d-%m-%Y'` : especifica que viene en formato día-mes-año
- Permite operaciones de fecha (extraer año, mes, día)

Códigos de formato:

Código	Significado	Ejemplo
<code>%d</code>	Día del mes (01-31)	15
<code>%m</code>	Mes (01-12)	01
<code>%Y</code>	Año con 4 dígitos	2021
<code>%H</code>	Hora (00-23)	14
<code>%M</code>	Minutos (00-59)	30

Salida:

```
=====
TRANSFORMACIÓN DE FECHAS
=====

Formato original: object
Ejemplo: 15-01-2021

☒ Formato transformado: datetime64[ns]
Ejemplo: 2021-01-15 00:00:00

Rango de fechas: 2021-01-08 00:00:00 a 2023-03-09 00:00:00
```

► 3.2 Crear columnas derivadas de fecha

Nueva celda:

```
# Extraer componentes de fecha
df_merged['year'] = df_merged['invoice_date'].dt.year
df_merged['month'] = df_merged['invoice_date'].dt.month
df_merged['day_of_week'] = df_merged['invoice_date'].dt.day_name()

print(' ☒ Columnas de fecha creadas:')
print(f'   - year: {df_merged["year"].unique()}')
print(f'   - month: {sorted(df_merged["month"].unique())}')
print(f'\nEjemplos de día de semana:')
print(df_merged[['invoice_date', 'day_of_week']].head(3))
```

Propiedades de `.dt` :

Propiedad	Retorna
<code>.dt.year</code>	Año (2021, 2022, ...)
<code>.dt.month</code>	Mes (1-12)
<code>.dt.day</code>	Día del mes (1-31)
<code>.dt.day_name()</code>	Nombre del día (Monday, Tuesday, ...)
<code>.dt.weekday</code>	Día de semana (0=Monday, 6=Sunday)
<code>.dt.quarter</code>	Trimestre (1-4)

Salida:

☒ Columnas de fecha creadas:

- year: [2021 2022 2023]
- month: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

Ejemplos de día de semana:

	invoice_date	day_of_week
0	2021-01-15	Friday
1	2021-01-15	Friday
2	2021-01-16	Saturday

► 3.3 Calcular total de venta

Nueva celda:

```
# Crear columna de total de venta
df_merged['total_sale'] = df_merged['quantity'] * df_merged['price']

print('☑ Columna total_sale creada')
print(f'\nEstadísticas de ventas:')
print(f'Total de ventas: ${df_merged["total_sale"].sum():,.2f}')
print(f'Venta promedio: ${df_merged["total_sale"].mean():,.2f}')
print(f'Venta mínima: ${df_merged["total_sale"].min():,.2f}')
print(f'Venta máxima: ${df_merged["total_sale"].max():,.2f}')
```

Salida:

```
☑ Columna total_sale creada

Estadísticas de ventas:
Total de ventas: $338,707,188.45
Venta promedio: $3,409.67
Venta mínima: $10.29
Venta máxima: $99,969.00
```

► 3.4 Crear grupos de edad

Nueva celda:

```
# Categorizar edades en grupos
def categorize_age(age):
    if age < 30:
        return 'Joven (18-29)'
    elif age < 50:
        return 'Adulto (30-49)'
    else:
        return 'Senior (50+)'

df_merged['age_group'] = df_merged['age'].apply(categorize_age)

print('☑ Grupos de edad creados')
print(f'\nDistribución:')
print(df_merged['age_group'].value_counts())
```

¿Qué hace `.apply()` ?

- Aplica una función a cada valor de la columna
- `categorize_age` es una función que creamos
- Retorna nueva columna con los resultados

Salida:

☒ Grupos de edad creados

Distribución:

Adulto (30-49) 46,182

Joven (18-29) 26,543

Senior (50+) 26,613

Name: age_group, dtype: int64



PASO 4: VALIDACIÓN FINAL

► 4.1 Verificar calidad del dataset limpio

Nueva celda:

```
# VALIDACIÓN FINAL
print('=' * 80)
print('VALIDACIÓN DE DATOS LIMPIOS')
print('=' * 80)

print(f'\n✅ RESUMEN FINAL:')
print(f'Total de registros: {len(df_merged):,}')
print(f'Total de columnas: {df_merged.shape[1]}')

print(f'\n📊 VERIFICACIÓN DE NULOS:')
nulos = df_merged.isnull().sum()
if nulos.sum() == 0:
    print('✅ No hay valores nulos en el dataset')
else:
    print(f'⚠️ Columnas con nulos:\n{nulos[nulos > 0]}')

print(f'\n📊 TIPOS DE DATOS:')
print(df_merged.dtypes)

print(f'\n✅ DATASET LISTO PARA ANÁLISIS')
```

Salida:

```
=====
VALIDACIÓN DE DATOS LIMPIOS
=====
```

```
☒ RESUMEN FINAL:
```

```
Total de registros: 99,338
```

```
Total de columnas: 16
```

```
 VERIFICACIÓN DE NULOS:
```

```
☒ No hay valores nulos en el dataset
```

```
 TIPOS DE DATOS:
```

```
transaction_id      int64
customer_id         int64
invoice_date        datetime64[ns]
invoice_no           object
product_category    object
quantity            int64
price               float64
payment_method      object
shopping_mall        object
age                 float64
gender              object
location            object
membership_years     int64
year                int64
month               int64
day_of_week         object
total_sale          float64
age_group           object
dtype: object
```

```
☒ DATASET LISTO PARA ANÁLISIS
```



PASO 5: GUARDAR DATOS LIMPIOS

► 5.1 Exportar a CSV

Nueva celda:

```
# GUARDAR DATOS LIMPIOS
import os

# Crear directorio si no existe
output_dir = '../datos'
os.makedirs(output_dir, exist_ok=True)

# Guardar CSV
output_path = os.path.join(output_dir, 'datos_limpios.csv')
df_merged.to_csv(output_path, index=False, encoding='utf-8')

print(f'✅ Dataset limpio guardado en: {output_path}')
print(f'📊 Tamaño del archivo: {os.path.getsize(output_path) / (1024*1024):.2f} MB')
print(f'📄 Total de registros: {len(df_merged):,}')
```

Parámetros de `.to_csv()` :

Parámetro	Descripción
<code>index=False</code>	No guardar índice del DataFrame
<code>encoding='utf-8'</code>	Codificación para caracteres especiales

Salida:

```
✅ Dataset limpio guardado en: ../datos/datos_limpios.csv
📊 Tamaño del archivo: 8.45 MB
📄 Total de registros: 99,338
```

PASO 6: COMMIT A GIT

```
# Guardar progreso
git add notebooks/analisis_etl.ipynb
git add datos/datos_limpios.csv
git commit -m "Implementar proceso ETL completo - datos limpios generados"
git push origin main
```

RESUMEN DEL PROCESO ETL

► Transformaciones aplicadas:

Paso	Acción	Resultado
1	Eliminar nulos en edad	99,338 registros (99.88%)
2	Fusionar datasets	16 columnas totales
3	Convertir fechas	Tipo <code>datetime64[ns]</code>
4	Extraer componentes fecha	<code>year</code> , <code>month</code> , <code>day_of_week</code>
5	Calcular total venta	<code>total_sale = quantity × price</code>
6	Categorizar edades	<code>age_group</code> (3 categorías)
7	Exportar datos limpios	<code>datos_limpios.csv</code> (8.45 MB)



COMANDOS PANDAS AVANZADOS APRENDIDOS

Comando	Función
<code>df.dropna(subset=['col'])</code>	Eliminar filas con nulos
<code>pd.merge(df1, df2, on='key')</code>	Fusionar DataFrames
<code>pd.to_datetime(df['col'], format='%d-%m-%Y')</code>	Convertir a fecha
<code>df['date'].dt.year</code>	Extraer año de fecha
<code>df['col'].apply(funcion)</code>	Aplicar función personalizada
<code>df.to_csv('archivo.csv')</code>	Exportar a CSV
<code>os.makedirs(path, exist_ok=True)</code>	Crear directorio



PRÓXIMOS PASOS

Ver **MÓDULO 4: ANÁLISIS EXPLORATORIO Y VISUALIZACIONES** para:

- Responder Items 4a-4e del TP
- Crear gráficos con Matplotlib
- Generar insights de negocio

Documento creado: 8 de noviembre de 2025

Parte de: Documentación Técnica Completa - Proyecto ETL TSCDIA